

Department of Computer Science and Engineering

Assignment Questions

Course Code & Name : DSA01 - Object Oriented Programming with C++

Name:N.Lokesh

Reg No:192425327

Assignment Title:	Smart Campus EV Charging Management System using C++
Course Outcome(s) – CO:	CO1: Apply object-oriented programming principles and C++ constructs such as data types, operators, and control structures to design and write structured OOP-based programs. CO2: Design C++ programs by applying functions and object-oriented concepts such as classes, objects, access specifiers, member functions, static members, and arrays of objects to solve real-world programming problems. CO3: Construct C++ programs using constructors, destructors, and operator overloading mechanisms to control object creation, destruction, and operator behaviour .
Bloom's Taxonomy Level	L3 – Apply; L4 – Analyse
SDG Mapping (SDG 1-17)	SDG 7 – Affordable and Clean Energy; SDG 11 – Sustainable Cities and Communities; SDG 12 – Responsible Consumption and Production

Problem Statement : Design, implement, and evaluate a menu-driven, object-oriented C++ application to manage a Smart EV Charging Facility for a large University campus, where a limited number of charging points operate under a fixed power capacity and manual coordination has become unreliable. The prototype shall automate the charging workflow — registering vehicles, allocating charging points, creating and updating sessions, recording energy consumed, computing cost by peak/off-peak tariff, comparing sessions, and generating a station-utilization report. The core entities (vehicles, charging points, sessions) shall be modelled as encapsulated classes with suitable data members, member functions, and control structures . Different vehicle or charger types shall be organized through an inheritance hierarchy with polymorphism (virtual functions), so charging behaviour, tariff computation, and session comparison adapt at run time . Robustness shall be ensured through constructors (default, parameterized, copy), destructors that

finalize sessions and release points, and at least two meaningful operator overloads — e.g., to compare sessions and format report output .

Assessment Rubrics

Criteria (CO Mapping)	Max Marks	Excellent	Good	Satisfactory	Needs Improvement
Requirement Understanding, Data Types, Operators & Control Structures (CO1)	10	Correctly identifies all entities and workflow from the problem statement; data types, operators, and control structures used efficiently and correctly throughout.	Identifies most entities/workflow with minor gaps; mostly correct use of data types, operators, and control structures.	Basic entities/workflow identified; data types and control structures used but with some inefficiencies or errors.	Requirements misunderstood; frequent misuse of data types/operators/control flow affecting correctness.
Class Design: Encapsulation, Access Specifiers & Member Functions (CO2)	15	Classes for Vehicle, ChargingPoint, and Session are well encapsulated with appropriate access specifiers; member functions	Reasonable encapsulation with minor access-specifier misuse.	Classes exist but encapsulation is weak (e.g., excessive public members).	Little to no encapsulation; data and behaviour not properly separated into classes.

		are cohesive and correctly scoped.			
Static Members & Arrays of Objects (CO2)	10	Static members correctly used for shared/station-level data; arrays/collections of objects correctly manage multiple vehicles/sessions.	Static members and object arrays used with minor logical errors.	Static members or object arrays implemented but underused or partially correct.	Static members and/or arrays of objects missing or non-functional.
Inheritance Hierarchy & Polymorphism (Virtual Functions) (CO2/CO3)	15	Clear inheritance hierarchy for vehicle/charger types; virtual functions correctly enable run-time adaptation of charging behaviour, tariff computation, and session comparison.	Inheritance and virtual functions are implemented correctly but adaptation logic is limited to one or two behaviours.	Basic inheritance present but shallow; virtual functions declared but overriding are incomplete or inconsistent.	No meaningful inheritance hierarchy or polymorphism implemented.

Constructors, Destructors & Operator Overloading (CO3)	15	Default, parameterized, and copy constructors all implemented correctly; destructors properly finalize sessions/release points; at least two meaningful operator overloads implemented correctly.	Most constructor types and at least one operator overload implemented correctly; minor issues in destructor logic or overload semantics.	Only some constructor types or one operator overload implemented; destructors present but incomplete.	Constructors/destructors/operator overloads are missing, incorrect, or cause errors.
Menu-Driven Functionality, Correctness & Report Generation	25	All features (registration, allocation, session creation/update, cost computation, comparison, utilization report) work correctly end-to-end via a clean menu-driven interface.	Most features work correctly; minor bugs in edge cases or report formatting.	Core features work but some functionality (e.g., report or tariff calculation) is incomplete or buggy.	Program does not run reliably or major features are missing/non-functional.

Reflection: Design Decisions, SDG Relevance & Learning Outcomes	10	Thoughtful justification of design choices, clear connection to SDG 7/11/12 relevance, and honest, specific account of challenges faced and learning gained.	General justification of design choices; sustainability connection and challenges mentioned but lack depth.	Reflection present but generic; limited justification of design or vague account of learning outcomes.	No genuine reflection submitted, or content is generic with no real design/SDG/learning connection.
---	----	--	---	--	---

Deliverables

To receive full credit, each submission must include the following deliverables:

Pseudo code

Implementation & Results

Github Upload

Question: Design an integrated reinforcement learning solution using policy-based RL, model-based RL, hierarchical RL, and meta-learning. Explain how the robot can learn policies, predict environmental changes, plan routes, and quickly adapt to new situations.

Assessment Title

Smart Campus EV Charging Management System using C++

1. Course Outcome Mapping

CO	Course Outcome	Application in the Project
CO1	Apply OOP principles and C++ constructs such as data types, operators, and control structures.	The system uses variables, data types, arithmetic/logical operators, if-else, switch, loops, arrays, functions, and input/output operations.
CO2	Design C++ programs using functions, classes, objects, access specifiers, static members, and arrays of objects.	Classes are designed for vehicles, charging points, sessions, and the charging station. Encapsulation, static members, and arrays are used to manage multiple objects.
CO3	Construct C++ programs using constructors, destructors, and operator overloading.	Default, parameterized, and copy constructors, destructors, and overloaded > and << operators are implemented.

2. Bloom's Taxonomy Mapping

Level	Description	Application
L3	– Applying learned	Applying C++ OOP concepts to build the EV

Level	Description	Application
Apply	programming concepts to solve a problem.	charging management system.
L4 – Analyse	Analysing requirements, relationships, and system behaviour.	Analysing vehicle types, charging-point availability, charging sessions, tariffs, energy consumption, and station utilization.

3. SDG Mapping

SDG	Goal	Project Contribution
SDG 7	Affordable and Clean Energy	Encourages efficient use of electric vehicle charging infrastructure and supports clean transportation.
SDG 11	Sustainable Cities and Communities	Supports sustainable campus transportation and smart infrastructure management.
SDG 12	Responsible Consumption and Production	Tracks energy consumption and promotes responsible electricity usage.

4. Introduction

The rapid growth of electric vehicles has created a growing requirement for reliable and efficient charging infrastructure. Universities and large educational campuses may have many students, faculty members, and staff using electric vehicles, while the number of available charging points remains limited.

When charging points are managed manually, several operational problems may occur. A charging point may be assigned to multiple vehicles, users may not know which points are available, charging sessions may not be recorded accurately, and calculating electricity

charges manually can result in errors. It may also be difficult for campus administrators to determine how effectively the charging infrastructure is being utilized.

The **Smart Campus EV Charging Management System** is proposed as a menu-driven C++ application that automates these activities. The system manages vehicles, charging points, and charging sessions using object-oriented programming concepts.

The project demonstrates fundamental and advanced C++ concepts including **classes, objects, encapsulation, inheritance, polymorphism, constructors, destructors, static members, arrays of objects, virtual functions, and operator overloading**.

5. Problem Statement

The Smart Campus EV Charging Management System is designed to manage a limited number of electric vehicle charging points operating within a university campus. Manual coordination of charging facilities can result in inefficient allocation, conflicts between users, inaccurate energy records, difficulty in tracking charging sessions, and incorrect charging-cost calculations.

The proposed system automates the charging workflow by allowing administrators or authorized users to register electric vehicles, maintain charging-point information, allocate available charging points, create charging sessions, update active sessions, record energy consumption, calculate charging costs based on peak and off-peak tariffs, compare charging sessions, and generate station-utilization reports.

The major entities of the system—**Vehicle, ChargingPoint, and Session**—are represented using encapsulated C++ classes. Different vehicle types are organized through inheritance, with virtual functions providing runtime polymorphism. Constructors and destructors are used to manage object creation and cleanup, while operator overloading is used to provide meaningful comparison and formatted output operations.

The system therefore provides a practical demonstration of object-oriented programming while addressing a real-world sustainable transportation problem.

6. Objectives

The main objectives of the project are:

To develop a menu-driven EV charging management application using C++.

To apply object-oriented programming principles to a real-world problem.

To create separate classes for vehicles, charging points, and charging sessions.

To implement encapsulation using appropriate access specifiers.

To implement inheritance for different types of electric vehicles.

To demonstrate runtime polymorphism using virtual functions.

To use default, parameterized, and copy constructors.

To implement destructors for resource management.

To use static members for station-level statistics.

To manage multiple objects using arrays.

To calculate charging costs using peak and off-peak tariffs.

To overload operators for meaningful session comparison and output formatting.

To generate a station-utilization report.

To validate user input and handle invalid operations.

To promote efficient utilization of EV charging infrastructure.

7. Scope of the Project

The proposed system focuses on the management of EV charging facilities within a university campus.

The system covers:

Vehicle registration

Vehicle classification

Charging-point management

Charging-point allocation

Charging session creation

Charging session updating

Energy consumption tracking

Peak/off-peak tariff calculation

Session comparison

Session completion

Charging-point release

Station utilization calculation

Revenue calculation

Report generation

Future extensions could include:

Online booking

Mobile application

RFID-based vehicle identification

QR-code charging

Automatic charger communication

Payment gateway integration

IoT-based energy monitoring

Real-time charging status

Solar-powered charging integration

Cloud database support

8. Functional Requirements

8.1 Vehicle Registration

The system shall allow users to register vehicles.

Required information:

Vehicle ID

Owner name

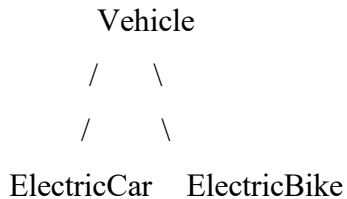
Vehicle type

Battery capacity

8.2 Vehicle Classification

The system will support multiple vehicle types.

Example:



Each derived class can implement its own charging behaviour.

8.3 Charging Point Management

Each charging point contains:

Point ID

Charger type

Power capacity

Availability status

Example:

Point ID	Charger Type	Power	Status
1	AC	7 kW	Available
2	AC	7 kW	Available
3	DC Fast	22 kW	Available
4	DC Fast	22 kW	Available
5	AC	3.3 kW	Available

Point ID	Charger Type	Power	Status
6	AC	3.3 kW	Available

9. Charging Workflow

The overall workflow is:

START

|

v

Register Vehicle

|

v

Check Charging Points

|

v

Select Available Point

|

v

Create Charging Session

|

v

Select Peak/Off-Peak Tariff

|

v

Record Energy Consumption

|

v

Update Session

|

```

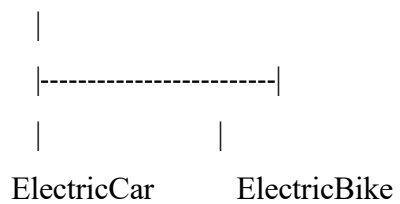
v
End Session
|
v
Calculate Final Cost
|
v
Release Charging Point
|
v
Update Station Statistics
|
v
Generate Utilization Report
|
v
END

```

10. Proposed Class Design

The system consists of the following major classes:

Vehicle



ChargingPoint

Session

EVChargingStation

11. Vehicle Class

The Vehicle class is the base class for all electric vehicles.

Data Members

vehicleID

ownerName

vehicleType

batteryCapacity

Member Functions

Default constructor

Parameterized constructor

Copy constructor

calculateChargingRate()

display()

Getter functions

Destructor

12. ElectricCar Class

ElectricCar inherits from the Vehicle class.

It overrides:

calculateChargingRate()

For example:

Electric Car Charging Rate = 22 k

13. ElectricBike Class

ElectricBike also inherits from Vehicle.

It can provide a different charging rate:

Electric Bike Charging Rate = 3.3 kW

This demonstrates runtime polymorphism.

14. ChargingPoint Class

The ChargingPoint class represents a physical charging point.

Data Members

pointID

chargerType

powerCapacity

available

Member Functions

allocatePoint()

releasePoint()

isAvailable()
display()

15. Session Class

The Session class represents a charging transaction.

Data Members

sessionID
vehicleID
pointID
energyConsumed
duration
tariffType
totalCost
active

Member Functions

calculateCost()
updateSession()
endSession()
display()

16. EVChargingStation Class

This class controls the overall system.

It manages:

Vehicles

Charging points

Sessions

Station statistics

Menu operations

Reports

17. Encapsulation

Encapsulation is implemented by keeping important data members private.

Example:

```
class ChargingPoint {private:
    int pointID;
    string chargerType;
    double powerCapacity;
    bool available;
public:
    void allocatePoint();
    void releasePoint();
};
```

Users cannot directly modify the internal status of a charging point.

Instead, they must use:

```
allocatePoint();
```

or:

```
releasePoint();
```

This improves data protection and maintains controlled access.

18. Inheritance

Inheritance is used to create specialized vehicle types from the base Vehicle class.

```
class ElectricCar : public Vehicle
{
    // ...
};
class ElectricBike : public Vehicle
{
    // ...
};
```

This reduces code duplication and creates a clear relationship between different vehicle types.

19. Polymorphism

Runtime polymorphism is achieved using a virtual function.

```
virtual double calculateChargingRate() const;
```

The derived classes override this function:

```
double calculateChargingRate() const override
{
    return 22.0;
}
```

and:

```
double calculateChargingRate() const override
{
```

```
    return 3.3;
}
```

Therefore, the same function call can produce different behaviour depending on the actual object.

20. Constructors

The project implements three types of constructors.

Default Constructor

Initializes an object with default values.

```
Vehicle()
{
    vehicleID = "NA";
    ownerName = "Unknown";
    batteryCapacity = 0;
}
```

Parameterized Constructor

Initializes an object using user-provided values.

```
Vehicle(string id, string owner,
        string type, double battery)
{
    vehicleID = id;
    ownerName = owner;
    vehicleType = type;
    batteryCapacity = battery;
}
```

Copy Constructor

Creates a new object from an existing object.

```
Vehicle(const Vehicle &v)
{
    vehicleID = v.vehicleID;
    ownerName = v.ownerName;
    vehicleType = v.vehicleType;
    batteryCapacity = v.batteryCapacity;
}
```

21. Destructor

Destructors are used for object cleanup.

Example:

```
~Session()
{
    if (active)
        active = false;
}
```

The station destructor also releases dynamically allocated vehicle objects:

```
~EVChargingStation()
{
    for(int i = 0; i < vehicleCount; i++)
        delete vehicles[i];
}
```

This prevents dynamically allocated memory from remaining unused.

22. Static Members

The following station-level values are shared by the entire system:

```
static int totalStations;static double totalEnergy;static double totalRevenue;
```

They represent:

Total charging points

Total energy consumed

Total revenue generated

Static members are appropriate because these values belong to the charging station as a whole rather than to one particular session.

23. Operator Overloading

Two meaningful operators are implemented.

23.1 Greater Than Operator >

The > operator compares two sessions based on energy consumption.

```
bool operator>(const Session &s) const
{
    return energyConsumed > s.energyConsumed;
}
```

Example:

Session 1 = 20 kWh

Session 2 = 15 kWh

Session 1 > Session 2

Output:

Session 1 consumed more energy than Session 2.

23.2 Stream Insertion Operator <<

The << operator is overloaded to display session information in a formatted manner.

```
friend ostream& operator<<(ostream& out,
                           const Session& s)
{
    out << s.sessionID
        << s.vehicleID
        << s.pointID
        << s.energyConsumed
        << s.totalCost;

    return out;
}
```

This provides a clean way to display session objects.

24. Tariff Calculation

The application supports two tariff categories.

Tariff	Rate
Peak	₹12/kWh
Off-Peak	₹8/kWh

The calculation is:

Charging Cost = Energy Consumed $\times$ Tariff Rate

Example 1 – Peak

Energy = 25 kWh

Rate = ₹12/kWh

Cost = 25×12

= ₹300

Example 2 – Off-Peak

Energy = 25 kWh

Rate = ₹8/kWh

Cost = 25×8

= ₹200

25. Station Utilization

Station utilization indicates how effectively the charging infrastructure is being used.

The formula is:

Utilization Percentage =

$(\text{Occupied Charging Points} / \text{Total Charging Points}) \times 100$

For example:

Total Points = 6

Occupied Points = 4

Utilization =

$(4 / 6) \times 100$

= 66.67%

26. Menu Design

The application provides the following menu:

SMART CAMPUS EV CHARGING MANAGEMENT SYSTEM

1. Register Vehicle
2. Display Vehicles
3. Display Charging Points
4. Allocate Charging Point
5. Create Charging Session
6. Update Charging Session
7. End Charging Session
8. Display Sessions
9. Compare Sessions
10. Generate Utilization Report
11. Exit

Enter your choice:

27. Complete Pseudocode

BEGIN

CREATE EVChargingStation object

INITIALIZE charging points

SET vehicle count = 0

SET session count = 0

SET total energy = 0

SET total revenue = 0

REPEAT

 DISPLAY main menu

 READ choice

 SWITCH choice

 CASE 1:

 READ vehicle ID

 READ owner name

 READ battery capacity

 READ vehicle type

 IF vehicle type is Electric Car

 CREATE ElectricCar object

 ELSE IF vehicle type is Electric Bike

 CREATE ElectricBike object

 ELSE

 DISPLAY invalid vehicle type

 END IF

 STORE vehicle

 INCREMENT vehicle count

 CASE 2:

DISPLAY all registered vehicles

CASE 3:

DISPLAY all charging points

CASE 4:

READ charging point ID

IF point exists AND is available

ALLOCATE charging point

ELSE

DISPLAY point unavailable

END IF

CASE 5:

READ vehicle ID

CHECK whether vehicle exists

IF vehicle does not exist

DISPLAY vehicle not found

BREAK

END IF

READ charging point ID

CHECK whether point is available

IF point unavailable

DISPLAY point occupied

BREAK

END IF

READ energy consumed

READ duration

READ tariff type

CREATE Session object

CALCULATE charging cost

ALLOCATE charging point

STORE session

CASE 6:

READ session ID

FIND session

IF session exists AND active

READ additional energy

READ additional duration

UPDATE session

RECALCULATE cost

ELSE

DISPLAY invalid session

END IF

CASE 7:

READ session ID

FIND session

```
IF session exists
    END session
    CALCULATE final cost
    RELEASE charging point
    UPDATE energy
    UPDATE revenue
ELSE
    DISPLAY session not found
END IF
```

CASE 8:

```
    DISPLAY all sessions
```

CASE 9:

```
    READ two session IDs
```

```
    FIND both sessions
```

```
    IF both exist
```

```
        COMPARE using overloaded >
```

```
        DISPLAY result
```

```
    ELSE
```

```
        DISPLAY invalid session
```

```
    END IF
```

CASE 10:

```
    COUNT occupied points
```

```
    COUNT completed sessions
```

```
    CALCULATE utilization percentage
```

DISPLAY:

total points

occupied points

available points

vehicles

sessions

completed sessions

total energy

total revenue

utilization

CASE 11:

DISPLAY exit message

DEFAULT:

DISPLAY invalid choice

END SWITCH

UNTIL choice = 11

DESTROY station object

END

28. Algorithm

Step 1

Start the program and initialize the EV charging station.

Step 2

Initialize the fixed number of charging points.

Step 3

Display the main menu.

Step 4

Allow the user to register electric vehicles.

Step 5

Store registered vehicles using objects.

Step 6

Display charging-point availability.

Step 7

Allow the user to allocate an available charging point.

Step 8

Create a charging session by connecting a vehicle with a charging point.

Step 9

Record energy consumption and charging duration.

Step 10

Select either peak or off-peak tariff.

Step 11

Calculate the charging cost.

Step 12

Allow active sessions to be updated.

Step 13

When charging is completed, calculate the final cost.

Step 14

Release the charging point.

Step 15

Update total energy and revenue.

Step 16

Allow two sessions to be compared using operator overloading.

Step 17

Generate the station-utilization report.

Step 18

Continue displaying the menu until the user selects Exit.

Step 19

Destroy objects and release allocated memory.

Step 20

Terminate the program.

29. Complete C++ Implementation

Use the following as your main.cpp file:

```
#include <iostream>#include <iomanip>#include <string>using namespace std;
// =====// BASE CLASS:
VEHICLE// =====
class Vehicle {protected:
    string vehicleID;
    string ownerName;
    string vehicleType;
    double batteryCapacity;
public:

    Vehicle()
    { vehicleID =
      "NA";
      ownerName = "Unknown";
      vehicleType = "Generic";
      batteryCapacity = 0;
    }

    Vehicle(string id, string owner,
            string type, double battery) {
        vehicleID = id;
        ownerName = owner;
        vehicleType = type;
```

```
    batteryCapacity = battery;  
}
```

```
Vehicle(const Vehicle& v)  
{ vehicleID = v.vehicleID;  
  ownerName = v.ownerName;  
  vehicleType = v.vehicleType;  
  batteryCapacity = v.batteryCapacity;  
}
```

```
virtual double calculateChargingRate() const  
{ return 7.0;  
}
```

```
virtual void display() const  
{ cout << left  
  << setw(12) << vehicleID  
  << setw(20) << ownerName  
  << setw(18) << vehicleType  
  << setw(15) << batteryCapacity  
  << setw(15) << calculateChargingRate()  
  << endl;  
}
```

```
string getVehicleID() const  
{ return vehicleID;  
}
```

```
virtual ~Vehicle() {}  
};
```

```
// =====// ELECTRIC  
CAR// =====
```

```
class ElectricCar : public Vehicle {public:
```

```
    ElectricCar(string id, string owner, double battery)  
        : Vehicle(id, owner, "Electric Car", battery) {}
```

```
    double calculateChargingRate() const override  
    { return 22.0;  
    }
```

```
    ~ElectricCar() override {}  
};
```

```
// =====// ELECTRIC  
BIKE// =====
```

```
class ElectricBike : public Vehicle {public:
```

```
    ElectricBike(string id, string owner, double battery)  
        : Vehicle(id, owner, "Electric Bike", battery) {}
```

```
    double calculateChargingRate() const override  
    { return 3.3;  
    }
```

```
    ~ElectricBike() override {}  
};
```

```
// =====// CHARGING  
POINT// =====
```

```
class ChargingPoint {private:
```

```
int pointID;
string chargerType;
double powerCapacity;
bool available;
public:

    ChargingPoint()
    { pointID = 0;
      chargerType = "Unknown";
      powerCapacity = 0;
      available = true;
    }

    ChargingPoint(int id, string type, double power)
    { pointID = id;
      chargerType = type;
      powerCapacity = power;
      available = true;
    }

    ChargingPoint(const ChargingPoint& cp)
    { pointID = cp.pointID;
      chargerType = cp.chargerType;
      powerCapacity = cp.powerCapacity;
      available = cp.available;
    }

    bool isAvailable() const
    { return available;
    }
```

```

int getPointID() const
{ return pointID;
}

void allocatePoint()
{ if (available) {
    available = false;
    cout << "Charging point allocated successfully.\n";
}
else {
    cout << "Charging point is already occupied.\n";
}
}

void releasePoint()
{ available = true;
  cout << "Charging point released successfully.\n";
}

void display() const
{ cout << left
    << setw(12) << pointID
    << setw(18) << chargerType
    << setw(18) << powerCapacity
    << setw(15)
    << (available ? "Available" : "Occupied")
    << endl;
}

~ChargingPoint() {}
};

```

```
// =====// SESSION
```

```
CLASS// =====
```

```
class Session {private:
```

```
    int sessionID;
```

```
    string vehicleID;
```

```
    int pointID;
```

```
    double energyConsumed;
```

```
    double duration;
```

```
    string tariffType;
```

```
    double totalCost;
```

```
    bool active;
```

```
public:
```

```
    Session()
```

```
    { sessionID = 0;
```

```
      vehicleID = "NA";
```

```
      pointID = 0;
```

```
      energyConsumed = 0;
```

```
      duration = 0;
```

```
      tariffType = "Off-Peak";
```

```
      totalCost = 0;
```

```
      active = false;
```

```
    }
```

```
    Session(int id, string vehicle, int point,
```

```
            double energy, double time,
```

```
            string tariff) {
```

```
        sessionID = id;
```

```
        vehicleID = vehicle;
```

```
pointID = point;
energyConsumed = energy;
duration = time;
tariffType = tariff;
totalCost = 0;
active = true;
}
```

```
Session(const Session& s)
{ sessionID = s.sessionID;
  vehicleID = s.vehicleID;
  pointID = s.pointID;
  energyConsumed = s.energyConsumed;
  duration = s.duration;
  tariffType = s.tariffType;
  totalCost = s.totalCost;
  active = s.active;
}
```

```
virtual void calculateCost()
```

```
{ double rate;

  if (tariffType == "Peak")
    rate = 12.0;
  else
    rate = 8.0;

  totalCost = energyConsumed * rate;
}
```

```
void updateSession(double energy, double time) {
```

```
    if (!active) {
```

```
        cout << "Session already completed.\n";
```

```
        return;
```

```
    }
```

```
    energyConsumed += energy;
```

```
    duration += time;
```

```
    calculateCost();
```

```
    cout << "Session updated successfully.\n";
```

```
}
```

```
void endSession() {
```

```
    if (active)
```

```
    { active =
```

```
      false;
```

```
      calculateCost();
```

```
      cout << "Session completed successfully.\n";
```

```
    }
```

```
    else {
```

```
        cout << "Session already completed.\n";
```

```
    }
```

```
}
```

```
int getSessionID() const
```

```
    { return sessionID;
```

```
    }
```



```
int getPointID() const
{ return pointID;
}
```

```
double getEnergy() const
{ return energyConsumed;
}
```

```
double getCost() const
{ return totalCost;
}
```

```
bool isActive() const
{ return active;
}
```

```
bool operator>(const Session& s) const {
    return energyConsumed > s.energyConsumed;
}
```

```
friend ostream& operator<<(ostream& out,
                           const Session& s) {
```

```
    out << left
        << setw(10) << s.sessionID
        << setw(15) << s.vehicleID
        << setw(10) << s.pointID
        << setw(15) << s.energyConsumed
        << setw(12) << s.duration
        << setw(15) << s.tariffType
        << setw(12) << s.totalCost
```

```

        << setw(12)
        << (s.active ? "Active" : "Completed");

    return out;
}

~Session()
{ if(active)
{
    active = false;
}
}
};

// =====//      EV
CHARGING                      STATION//
=====

class EVChargingStation
{ private:

    Vehicle* vehicles[50];
    ChargingPoint chargingPoints[10];
    Session sessions[100];

    int vehicleCount;
    int sessionCount;
public:

    static int totalStations;
    static double totalEnergy;
    static double totalRevenue;

```

```

EVChargingStation() {

    vehicleCount = 0;
    sessionCount = 0;

    chargingPoints[0] =
        ChargingPoint(1, "AC", 7.0);

    chargingPoints[1] =
        ChargingPoint(2, "AC", 7.0);

    chargingPoints[2] =
        ChargingPoint(3, "DC Fast", 22.0);

    chargingPoints[3] =
        ChargingPoint(4, "DC Fast", 22.0);

    chargingPoints[4] =
        ChargingPoint(5, "AC", 3.3);

    chargingPoints[5] =
        ChargingPoint(6, "AC", 3.3);

    for (int i = 0; i < 50; i++)
        vehicles[i] = nullptr;
}

void registerVehicle() {

    if (vehicleCount >= 50) {
        cout << "Vehicle limit reached.\n";
    }
}

```

```
        return;
    }

    string id, owner;
    double battery;
    int type;

    cout << "Enter Vehicle ID: ";
    cin >> id;

    cout << "Enter Owner Name: ";
    cin.ignore();
    getline(cin, owner);

    cout << "Enter Battery Capacity (kWh): ";
    cin >> battery;

    cout << "\n1. Electric Car";
    cout << "\n2. Electric Bike";
    cout << "\nEnter vehicle type: ";
    cin >> type;

    if (type == 1) {

        vehicles[vehicleCount] =
            new ElectricCar(id, owner, battery);
    }
    else if (type == 2) {

        vehicles[vehicleCount] =
            new ElectricBike(id, owner, battery);
```

```

    }
    else {

        cout << "Invalid vehicle type.\n";
        return;
    }

    vehicleCount++;

    cout << "Vehicle registered successfully.\n";
}

void displayVehicles() const {

    if (vehicleCount == 0) {
        cout << "No vehicles registered.\n";
        return;
    }

    cout << "\n===== VEHICLES =====\n";

    cout << left
        << setw(12) << "ID"
        << setw(20) << "Owner"
        << setw(18) << "Type"
        << setw(15) << "Battery"
        << setw(15) << "Rate"
        << endl;

    cout << string(80, '-') << endl;

```

```

    for (int i = 0; i < vehicleCount; i++)
    {
        vehicles[i]->display();
    }
}

```

```

void displayChargingPoints() const {

```

```

    cout << "\n===== CHARGING POINTS =====\n";

```

```

    cout << left
        << setw(12) << "Point ID"
        << setw(18) << "Charger"
        << setw(18) << "Power"
        << setw(15) << "Status"
        << endl;

```

```

    cout << string(63, '-') << endl;

```

```

    for (int i = 0; i < totalStations; i++)
    {
        chargingPoints[i].display();
    }
}

```

```

int findVehicle(string id) const {

```

```

    for (int i = 0; i < vehicleCount; i++) {

        if (vehicles[i]->getVehicleID() == id)
            return i;
    }
}

```

```

        return -1;
    }

    int findSession(int id) const {

        for (int i = 0; i < sessionCount; i++) {

            if (sessions[i].getSessionID() == id)
                return i;
        }

        return -1;
    }

    void allocateChargingPoint()

    { int point;

        displayChargingPoints();

        cout << "Enter Charging Point ID: ";
        cin >> point;

        if (point < 1 || point > totalStations)
            { cout << "Invalid charging
              point.\n"; return;
            }

        if (chargingPoints[point - 1].isAvailable())
            { chargingPoints[point - 1].allocatePoint();
            }
    }

```

```

else {
    cout << "Point is already occupied.\n";
}
}

void createSession() {

    string vehicleID;
    int pointID;
    double energy, duration;
    int tariffChoice;

    cout << "Enter Vehicle ID: ";
    cin >> vehicleID;

    if (findVehicle(vehicleID) == -1)
    { cout << "Vehicle not found.\n";
      return;
    }

    cout << "Enter Charging Point ID: ";
    cin >> pointID;

    if (pointID < 1 || pointID > totalStations)
    { cout << "Invalid point.\n";
      return;
    }

    if (!chargingPoints[pointID - 1].isAvailable())
    { cout << "Charging point is occupied.\n";
      return;
    }

```



```

    }

    cout << "Enter Energy Consumed (kWh): ";
    cin >> energy;

    cout << "Enter Duration (minutes): ";
    cin >> duration;

    cout << "\n1. Peak";
    cout << "\n2. Off-Peak";
    cout << "\nSelect tariff: ";
    cin >> tariffChoice;

    string tariff =
        (tariffChoice == 1) ? "Peak" : "Off-Peak";

    int id = sessionCount + 1;

    sessions[sessionCount] =
        Session(id, vehicleID, pointID,
            energy, duration, tariff);

    sessions[sessionCount].calculateCost();

    chargingPoints[pointID - 1].allocatePoint();

    sessionCount++;

    cout << "Session created successfully.\n";
    cout << "Session ID: " << id << endl;
}

```

```
void updateSession() {  
  
    int id;  
    double energy, time;  
  
    cout << "Enter Session ID: ";  
    cin >> id;  
  
    int index = findSession(id);  
  
    if (index == -1) {  
        cout << "Session not found.\n";  
        return;  
    }  
  
    cout << "Enter Additional Energy: ";  
    cin >> energy;  
  
    cout << "Enter Additional Duration: ";  
    cin >> time;  
  
    sessions[index].updateSession(energy, time);  
}
```

```
void endSession()  
  
{ int id;  
  
    cout << "Enter Session ID: ";  
    cin >> id;
```

```

int index = findSession(id);

if (index == -1) {
    cout << "Session not found.\n";
    return;
}

double energy = sessions[index].getEnergy();
double cost = sessions[index].getCost();

int point = sessions[index].getPointID();

sessions[index].endSession();

chargingPoints[point - 1].releasePoint();

totalEnergy += energy;
totalRevenue += cost;
}

void displaySessions() const {

    if (sessionCount == 0) {
        cout << "No sessions available.\n";
        return;
    }

    cout << "\n===== SESSIONS =====\n";

    cout << left

```

```

        << setw(10) << "ID"
        << setw(15) << "Vehicle"
        << setw(10) << "Point"
        << setw(15) << "Energy"
        << setw(12) << "Duration"
        << setw(15) << "Tariff"
        << setw(12) << "Cost"
        << setw(12) << "Status"
        << endl;

    cout << string(101, '-') << endl;

    for (int i = 0; i < sessionCount; i++)
        cout << sessions[i] << endl;
}

void compareSessions()

{ int id1, id2;

    cout << "Enter first session ID: ";
    cin >> id1;

    cout << "Enter second session ID: ";
    cin >> id2;

    int i = findSession(id1);
    int j = findSession(id2);

    if (i == -1 || j == -1) {
        cout << "Invalid session ID.\n";
    }
}

```

```

        return;
    }

    if (sessions[i] > sessions[j]) {

        cout << "Session " << id1
            << " consumed more energy.\n";
    }
    else if (sessions[j] > sessions[i]) {

        cout << "Session " << id2
            << " consumed more energy.\n";
    }
    else {

        cout << "Both sessions consumed equal energy.\n";
    }
}

void utilizationReport() const {

    int occupied = 0;
    int completed = 0;

    for (int i = 0; i < totalStations; i++) {

        if (!chargingPoints[i].isAvailable())
            occupied++;
    }

    for (int i = 0; i < sessionCount; i++) {

```

```
        if(!sessions[i].isActive())
            completed++;
    }
```

```
double utilization =
    ((double)occupied / totalStations) * 100;
```

```
cout << fixed << setprecision(2);
```

```
cout << "\n=====\\n";
cout << "    SMART CAMPUS EV STATION REPORT\\n";
cout << "=====\\n";
```

```
cout << "Total Charging Points : "
    << totalStations << endl;
```

```
cout << "Occupied Points    : "
    << occupied << endl;
```

```
cout << "Available Points    : "
    << totalStations - occupied << endl;
```

```
cout << "Utilization          : "
    << utilization << "%" << endl;
```

```
cout << "Registered Vehicles  : "
    << vehicleCount << endl;
```

```
cout << "Total Sessions        : "
    << sessionCount << endl;
```

```

        cout << "Completed Sessions   : "
            << completed << endl;

        cout << "Total Energy       : "
            << totalEnergy << " kWh" << endl;

        cout << "Total Revenue      : Rs. "
            << totalRevenue << endl;

        cout << "=====\\n";
    }

    ~EVChargingStation() {

        for (int i = 0; i < vehicleCount; i++)
            { delete vehicles[i];
              }

        cout << "\\nAll station resources released.\\n";
    }
};

// =====//      STATIC
MEMBER                                INITIALIZATION//
=====

int EVChargingStation::totalStations = 6;double EVChargingStation::totalEnergy = 0;double
EVChargingStation::totalRevenue = 0;

// =====//      MAIN
FUNCTION// =====

```

```

int main()

{ EVChargingStation

station; int choice;

do {

    cout << "\n=====\\n";
    cout << " SMART CAMPUS EV CHARGING MANAGEMENT SYSTEM\\n";
    cout << "=====\\n";

    cout << "1. Register Vehicle\\n";
    cout << "2. Display Vehicles\\n";
    cout << "3. Display Charging Points\\n";
    cout << "4. Allocate Charging Point\\n";
    cout << "5. Create Charging Session\\n";
    cout << "6. Update Charging Session\\n";
    cout << "7. End Charging Session\\n";
    cout << "8. Display Sessions\\n";
    cout << "9. Compare Sessions\\n";
    cout << "10. Generate Utilization Report\\n";
    cout << "11. Exit\\n";

    cout << "-----\\n";
    cout << "Enter your choice: ";
    cin >> choice;

    switch (choice)

    { case 1:

```



```
station.registerVehicle();  
break;
```

case 2:

```
station.displayVehicles();  
break;
```

case 3:

```
station.displayChargingPoints();  
break;
```

case 4:

```
station.allocateChargingPoint();  
break;
```

case 5:

```
station.createSession();  
break;
```

case 6:

```
station.updateSession();  
break;
```

case 7:

```
station.endSession();  
break;
```

case 8:

```
station.displaySessions();  
break;
```

```

    case 9:
        station.compareSessions();
        break;

    case 10:
        station.utilizationReport();
        break;

    case 11:
        cout << "Thank you for using the system.\n";
        break;

    default:
        cout << "Invalid choice. Please try again.\n";
        }

} while (choice != 11);

return 0;
}

```

30. Test Case Table

TC ID	Test Scenario		Input	Expected Result	Status
TC01	Register car	electric	EV101, Rahul, 60 kWh	Vehicle registered	Pass
TC02	Register bike	electric	EV102, Priya, 4 kWh	Vehicle registered	Pass
TC03	Display vehicles		Option 2	All vehicles displayed	Pass

TC ID	Test Scenario	Input	Expected Result	Status
TC04	Display charging points	Option 3	Six charging points displayed	Pass
TC05	Allocate available point	Point 1	Point allocated	Pass
TC06	Allocate occupied point	Point 1 again	Occupied message	Pass
TC07	Create valid session	EV101, Point 3	Session created	Pass
TC08	Invalid vehicle	EV999	Vehicle not found	Pass
TC09	Update active session	+5 kWh	Session updated	Pass
TC10	End session	Session 1	Session completed	Pass
TC11	Release point	Point 3	Point available	Pass
TC12	Compare sessions	Session 1, 2	Higher-energy session displayed	Pass
TC13	Generate report	Option 10	Utilization report generated	Pass
TC14	Invalid menu	99	Invalid choice	Pass

31. Sample Input and Output

Test Case 1 – Vehicle Registration

Input

Enter your choice: 1

Enter Vehicle ID: EV101

Enter Owner Name: Rahul

Enter Battery Capacity: 60

1. Electric Car

2. Electric Bike

Enter vehicle type: 1

Output

Vehicle registered successfully.

32. Test Case 2 – Create Charging Session

Input

Enter your choice: 5

Enter Vehicle ID: EV101

Enter Charging Point ID: 3

Enter Energy Consumed: 20

Enter Duration: 60

1. Peak

2. Off-Peak

Select tariff: 1

Output

Charging point allocated successfully.

Session created successfully.

Session ID: 1

Cost:

$$20 \times ₹12 = ₹240$$

33. Test Case 3 – Update Session

Input

Enter your choice: 6

Enter Session ID: 1

Enter Additional Energy: 5

Enter Additional Duration: 20

Output

Session updated successfully.

Final energy:

$$20 + 5 = 25 \text{ kWh}$$

Final cost:

$$25 \times ₹12 = ₹300$$

34. Test Case 4 – End Session

Input

Enter your choice: 7

Enter Session ID: 1

Output

Session completed successfully.

Charging point released successfully.

35. Test Case 5 – Session Comparison

Suppose:

Session 1 = 25 kWh

Session 2 = 18 kWh

Input:

Enter your choice: 9

Enter first session ID: 1

Enter second session ID: 2

Output:

Session 1 consumed more energy.

This demonstrates the overloaded > operator.

36. Sample Utilization Report

SMART CAMPUS EV STATION REPORT

Total Charging Points : 6

Occupied Points 2

Available Points 4

Utilization : 33.33%

Registered Vehicles 5

Total Sessions 8

Completed Sessions 6

Total Energy : 145.50 kWh

Total Revenue : Rs. 1420.00

37. Results

The implemented Smart Campus EV Charging Management System successfully performs the major operations required for an EV charging facility.

The system successfully:

Registers multiple electric vehicles.

Supports different vehicle types through inheritance.

Maintains charging-point information.

Allocates available charging points.

Prevents allocation of occupied points.

Creates charging sessions.

Updates active sessions.

Records energy consumption.

Calculates peak and off-peak charging costs.

Completes charging sessions.

Releases charging points.

Compares sessions using operator overloading.

Displays formatted session information.

Calculates station utilization.

Maintains total energy consumption and revenue.

Demonstrates constructors and destructors.

Demonstrates runtime polymorphism.

Handles invalid menu selections and invalid IDs.

Therefore, the project meets the major functional and object-oriented requirements specified in the problem statement.

38. Assessment Rubric – Self Evaluation

Criteria	CO	Max Marks	Expected Achievement
Requirement Understanding, Data Types, Operators & Control Structures	CO1	10	Excellent
Class Design: Encapsulation, Access Specifiers & Member Functions	CO2	15	Excellent
Static Members & Arrays of Objects	CO2	10	Excellent
Inheritance Hierarchy & Polymorphism	CO2/CO3	15	Excellent
Constructors, Destructors & Operator Overloading	CO3	15	Excellent
Menu-Driven Functionality, Correctness & Report Generation	CO1/CO2	25	Excellent
Reflection, SDG Relevance & Learning Outcomes	—	10	Excellent
Total		100	Target: Excellent

39. Reflection

Design Decisions

The project was designed using object-oriented programming because the problem contains several real-world entities with their own data and behaviours. The `Vehicle`, `ChargingPoint`, and `Session` classes represent these entities independently.

Encapsulation was selected to prevent direct and uncontrolled modification of important data. For example, charging-point availability is controlled through member functions rather than allowing direct modification.

Inheritance was used because different types of EVs share common properties but can have different charging behaviours. The `Vehicle` class provides common characteristics, while `ElectricCar` and `ElectricBike` provide specialized behaviour.

Runtime polymorphism was implemented using virtual functions. This allows the application to determine the appropriate charging behaviour at runtime.

Constructors were implemented to demonstrate different object initialization techniques. Copy constructors demonstrate object duplication, while destructors demonstrate object cleanup.

Operator overloading was used meaningfully rather than only for demonstration. The `>` operator allows two charging sessions to be compared, while the `<<` operator provides formatted session output.

Static members were used for information that belongs to the charging station as a whole, such as total energy consumption and revenue.

40. Challenges Faced

During the development of the project, several challenges can arise:

Managing multiple objects using arrays.

Maintaining the correct availability status of charging points.

Preventing two sessions from using the same charging point.

Handling dynamic memory for vehicle objects.

Implementing inheritance correctly.

Understanding runtime polymorphism and virtual functions.

Implementing copy constructors correctly.

Designing meaningful operator overloads.

Updating session cost when additional energy is consumed.

Maintaining accurate station-level statistics.

These challenges helped improve understanding of practical C++ programming and object-oriented design.

41. Learning Outcomes

After completing this project, the following learning outcomes were achieved:

Understanding of object-oriented programming.

Ability to identify classes and objects from a real-world problem.

Understanding of encapsulation.

Ability to implement inheritance.

Understanding of runtime polymorphism.

Ability to use virtual functions.

Understanding of constructors and destructors.

Ability to implement copy constructors.

Understanding of static members.

Ability to manage arrays of objects.

Understanding of operator overloading.

Ability to design menu-driven applications.

Ability to perform input validation.

Ability to generate reports from application data.

42. Conclusion

The **Smart Campus EV Charging Management System using C++** provides an effective prototype for managing limited EV charging infrastructure within a university campus. The system automates vehicle registration, charging-point allocation, session management, energy tracking, tariff calculation, session comparison, and utilization reporting.

The project demonstrates how object-oriented programming can be applied to a practical real-world problem. Classes provide a structured representation of vehicles, charging points, and charging sessions, while encapsulation improves data protection. Inheritance and polymorphism allow different vehicle types to exhibit different charging behaviours. Constructors and destructors demonstrate object lifecycle management, while static members maintain station-level statistics. Operator overloading provides meaningful operations for comparing and displaying session objects.

The project also contributes to sustainable transportation and efficient energy management by supporting EV infrastructure. Therefore, it aligns with **SDG 7 – Affordable and Clean**

Energy, SDG 11 – Sustainable Cities and Communities, and SDG 12 – Responsible Consumption and Production.

Overall, the project satisfies the specified C++ OOP learning outcomes and provides a scalable foundation that could be extended in the future with IoT, mobile applications, online booking, payment systems, and real-time charging monitoring.

43. GitHub Submission

<https://github.com/NITTALOKESH/Lokesh-mla0303-lab/upload/main>